

Database Basics

 [_ \(https://github.com/learn-co-curriculum/python-p3-database-basics\)](https://github.com/learn-co-curriculum/python-p3-database-basics)  [_ \(https://github.com/learn-co-curriculum/python-p3-database-basics/issues/new\)](https://github.com/learn-co-curriculum/python-p3-database-basics/issues/new)

Learning Goals

- Use SQL to store data and retrieve it later on.
 - Use SQLite to build relational databases on your computer.
-

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
 - **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
 - **Query**: a statement used to return data from a database.
 - **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
 - **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
 - **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
 - **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.
-

Introduction

We'll cover how to create and delete database tables in SQLite as well as how to add columns to an existing table.

Database Structure

Relational Databases like SQLite store data in a structure we refer to as a table. You can think of a table in a database a lot like you would a spreadsheet. We define specific columns in our table, and then we store any number of what we refer to as 'records' as rows in our database. A record is just information referring to one specific entity. For instance, if you had a table called "People" you could imagine a structure like this:

name	age	email
Bob	29	bob@flatironschool.com (mailto:bob@flatironschool.com)
Avi	28	avi@flatironschool.com (mailto:avi@flatironschool.com)
Adam	28	adam@flatironschool.com (mailto:adam@flatironschool.com)

Each column has a name, and each row contains the corresponding information about a person.

Thinking to what you've learned already about object oriented Python, you can also imagine how we might connect the idea of a **table** in SQL to a **class** in Python, and a **record** within a table to an **instance** of a class, or **object**. For the example above, our Python representation of those same objects might look like this:

```
class Person:
    def __init__(self, name, age, email)
        self.name = name
        self.age = age
        self.email = email

bob = Person("Bob", 29, "bob@flatironschool.com")
avi = Person("Avi", 28, "avi@flatironschool.com")
adam = Person("Adam", 28, "adam@flatironschool.com")
```

► Would *age* be an example of a table, row, or column in SQL?

Note on Column Names

When we name columns in our database, there are a couple of conventions we will follow. The first is that we will always use **lowercase letters** when referring to column names in our database. SQLite isn't case sensitive about its commands or column names, but it is generally best practice for us to stick to lowercase for our column names.

The second convention we want to follow is more important. That is, when we have multiple words in a column name, we link them together using underscores rather than spaces. We call this convention **snake case**. So, for instance, if we wanted to be more specific with our email column above, we can name it email_address. If we wanted to split up name to first and last we might have columns called `first_name` and `last_name`.

Database Tables

In the following sections, we'll cover how to create, alter, and delete database tables. This reading is accompanied by a code along exercise that you can do in your terminal. You don't need to fork this repository, and there are no tests to pass. Follow along with the reading and code along instructions.

Create Table

When we create a new database, it comes like a sort of blank slate. We can then create a table inside our database using the following statement:

```
CREATE TABLE table_name;
```

But before we're able to store any actual data in a table, we'll need to define the columns in the table as well as the specific type of data each column will store.

Let's give it a shot. For the purposes of this code along, you'll be typing these commands into your terminal.

Code Along 1: Creating a Table

In the terminal let's create our new database and start `sqlite3` by running the following:

```
$ sqlite3 pet_database.db
```

Now, at our SQLite prompt, let's create our table:

```
CREATE TABLE cats;
```

You should see the following error:

```
Error: near ";": syntax error
```

SQLite expects us to include at least some definition of the structure of this table as well. In other words, when we create database tables, we need to specify some column names, along with the type of data we are planning to store in each column. More on data types later.

Let's try that table statement again:

```
CREATE TABLE cats (  
  id INTEGER PRIMARY KEY,  
  name TEXT,  
  age INTEGER  
);
```

Let's break down the above code:

1. Use the `CREATE TABLE` command to create a new table called "cats".
2. Include a list of column names along with the type of data they will be storing. `TEXT` means we'll be storing plain old text, `INTEGER` means we'll store a number. Note that the use of capitalization is arbitrary, but it is a convention to help separate the SQL commands from the names we make up for our tables and columns.
3. Every table we create, regardless of the other column names and data types, should be defined with an `id INTEGER PRIMARY KEY` column, including the integer data type and primary key designation. Our SQLite database tables *must be indexed by a number*. We want each row in our table to have a number, which we'll call "id", just like in an Excel spreadsheet. Numbering our table rows makes our data that much easier

to access, update, and organize. SQLite comes with a data type designation called "Primary Key". Primary keys are unique and auto-incrementing, meaning they start at 1 and each new row automatically gets assigned the next numeric value.

Okay, let's check and make sure that we successfully created that table. To do this we'll be using SQL commands. To get a complete list of commands, you can type `.help` into the sqlite prompt.

```
sqlite> .help
.auth ON|OFF          Show authorizer callbacks
.backup ?DB? FILE      Backup DB (default "main") to FILE
.bail on|off           Stop after hitting an error.  Default OFF
.binary on|off         Turn binary output on or off.  Default OFF
.cd DIRECTORY          Change the working directory to DIRECTORY
.changes on|off        Show number of rows changed by SQL
```

Wow, that's a lot. Don't worry too much about all of these different commands right now. Just know that you can always use `.help` to check out the available options.

Okay, let's check out our new table. To list all the tables in the database we'll use the `.tables` command. Type it into the sqlite prompt and hit enter, and you should see our `cats` table listed.

We can look at the structure, or "schema", of our database (i.e. the tables and their columns + column data types) with the `.schema` command:

```
sqlite> .schema
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER
);
```

You can also use the SQLite VSCode extension, or DB Browser for SQLite, to see a visual representation of the table. There won't be much to look at yet, since we haven't added any data to the table; but you will be able to see the structure of the table.

Let's move on to altering our table.

Alter Table

Let's say that, after creating a database and creating a table to live inside that database, we decide we want to add or remove a column. We can do so with the `ALTER TABLE` statement.

Code Along 2: Adding, Removing and Renaming Columns

Let's say we want to add a new column, `breed`, to our `cats` table:

```
ALTER TABLE cats ADD COLUMN breed TEXT;
```

Let's check out our schema now:

```
sqlite> .schema
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  breed TEXT
);
```

Notice that the `ALTER` statement isn't here, but instead SQLite has updated our original `CREATE` statement. The schema reflects the current structure of the database, which is reflected as the `CREATE` statement necessary to create that structure.

Unfortunately, altering a column name and/or deleting a column can be tricky in SQLite3. There are workarounds, however. We're not going to get into that right now, but you can explore the [documentation on this topic](https://www.sqlite.org/lang_altertable.html)  [_\(https://www.sqlite.org/lang_altertable.html\)_](https://www.sqlite.org/lang_altertable.html).

Fortunately, SQLite still supports most of what we'll need one way or another. For now, if you need to change a column name, it's best to simply delete the table and re-create it.

Drop Table

Lastly, we'll discuss how to delete a table from a database with the `DROP TABLE` statement.

Code Along 3: Deleting a Table

Deleting a table is very simple:


```
DROP TABLE cats;
```

And that's it! You can exit out of the sqlite prompt with the `.quit` command.

Resources

- [SQL Tutorial - W3Schools](https://www.w3schools.com/sql/)  [\(https://www.w3schools.com/sql/\)](https://www.w3schools.com/sql/)
- [Documentation - SQLite](https://www.sqlite.org/docs.html)  [\(https://www.sqlite.org/docs.html\)](https://www.sqlite.org/docs.html)
- [SQLite - VisualStudio Marketplace](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)  [\(https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite\)](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)
- [SQLite Keywords - SQLite](https://www.sqlite.org/lang_keywords.html)  [\(https://www.sqlite.org/lang_keywords.html\)](https://www.sqlite.org/lang_keywords.html)
- [ZetCode sqlite3 Tutorial](http://zetcode.com/db/sqlite/)  [\(http://zetcode.com/db/sqlite/\)](http://zetcode.com/db/sqlite/)